

AI Critique

Gemini helped turn the assignment constraints into an initial workload model, but its first answer mixed source facts, generic testing patterns, and unsupported predictions. It claimed `SQLITE_BUSY` would be the primary bottleneck and that Node.js would saturate one core first. The source actually opens one shared `sqlite3` connection, so request queuing was at least as plausible. The measured Stress run produced no lock errors; instead, throughput changed only from 324.06 to 326.86 samples/s between 30 and 40 users while p95 rose from 74 to 109 ms. Gemini also inherited my incorrect statement that duplicate emails fail. Source review and two real requests proved the opposite: both returned HTTP 200 with different IDs. Its proposal for 50,000 CSV rows and replacement of an open database file was unnecessary and unsafe. A UUID email and controlled backend restart were more defensible.

The raw-JTL interaction exposed a second failure mode. Although four complete files were attached, Gemini saw truncated strings and estimated about 380 Load and 260 Stress samples instead of 19,311 and 139,140. A corrective prompt supplied deterministic nearest-rank results, which Gemini reproduced, but it then attributed the capacity knee to a "fully saturated" internal component and claimed that 15-minute telemetry proved no memory leak. A final challenge made it retract both claims and limit the conclusion to the observation window. The revised design also replaced substring checks with parsed JSON matching the ID and email on the same object. These errors show why fluent caveats are not enough: provenance, arithmetic, and causal claims require separate checks. My reliable pattern became propose, inspect source, execute, recalculate from raw rows, challenge with exact counter-evidence, and accept only conclusions bounded by the measurement.

Word count: **279** (body only, whitespace-separated count).